Monte Carlo Simulation for European Option Pricing

Project Report - I

Submitted for the partial fulfillment of the degree of

Bachelor of Technology

In

Mathematics and Computing

Submitted By

Rudransh Shukla (BTMC25O1119)

Vaibhav Sharma (BTMC25O1144)

UNDER THE SUPERVISION AND GUIDANCE OF

Dr. Dheeraj Kumar Dixit

Assistant Professor & Head

Department of Engineering Mathematics & Computing



माधव प्रौद्योगिकी एवं विज्ञान संस्थान, ग्वालियर (म.प्र.), भारत

MADHAV INSTITUTE OF TECHNOLOGY & SCIENCE, GWALIOR (M.P.), INDIA

Deemed University

(Declared under Distinct Category by Ministry of Education, Government of India)

NAAC ACCREDITED WITH A++ GRADE

July-December 2026

DECLARATION BY THE CANDIDATE

We hereby declare that the work entitled "Monte Carlo Simulation for European Option Pricing" is our work, conducted under the supervision of Dr. Dheeraj Kumar Dixit, Assistant Professor & Head, during the session July-December 2026. The report submitted by us is a record of bonafide work carried out by us.

We further declare that the work reported in this report has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Rudransh Shukla
Enrolment No.: BTMC25O1119
Department of Engineering Mathematics & Computing

Vaibhav Sharma
Enrolment No.: BTMC25O1144
Department of Engineering Mathematics & Computing

Date: 27 October 2026
Place: Gwalior

This is to certify that the above statement made by the candidates is correct to the best of my knowledge and belief.

Guided By:

Dr. Dheeraj Kumar Dixit
Assistant Professor & Head
Department of Engineering Mathematics & Computing
MITS DU Gwalior

Departmental Project Coordinator

Approved by HoD

Dr. Dheeraj Kumar Dixit
Assistant Professor & Head

Dr. Kuldeep K. Tiwari
MAC Second Year Project Coordinator

Dr. Monika Dandotiya
MAC Second Year Project Coordinator

ABSTRACT

Option pricing is a fundamental problem in financial mathematics. A European option gives its holder the right to buy (call) or sell (put) an asset at a fixed strike price on a fixed expiry date, and its fair price is the discounted expected payoff under risk-neutral valuation. This project implements a Monte Carlo simulation to price European call and put options using Python and NumPy.

The report first develops the required background at an introductory level: probability and statistics (distributions and the Central Limit Theorem), stochastic processes (the Markov property, random walk, Brownian motion, the lognormal distribution, Geometric Brownian Motion and an intuitive form of Itô's lemma) and financial mathematics (continuous compounding, option payoffs, risk-neutral valuation and the Black–Scholes formula). The Monte Carlo theory — the Law of Large Numbers, pseudo-random number generation with a linear congruential generator, the Box–Muller transform, Euler–Maruyama discretisation, standard error and confidence intervals — is then linked to short Python fragments that combine into a complete vectorised pricer.

For $S_0 = 100$, $K = 100$, $r = 5\%$, $\sigma = 20\%$ and $T = 1$ year, a simulation of 100,000 paths gives a call price of 10.4205 and a put price of 5.6122, compared with the Black–Scholes values 10.4506 and 5.5735; both exact values lie inside the 95% confidence intervals. The standard error decreases proportionally to $1/\sqrt{N}$, the confidence intervals show 95.3% coverage, NumPy vectorisation gives a speed-up of about 24 times over a loop, and antithetic variates halve the variance of the estimate.

Keywords: Monte Carlo simulation, European options, Black–Scholes model, Geometric Brownian Motion, risk-neutral valuation, NumPy, variance reduction

ACKNOWLEDGEMENT

We express our sincere gratitude to our Hon'ble Vice Chancellor, Dr. R. K. Pandit, for providing the academic environment and resources necessary for the successful completion of this project.

We are equally thankful to the Pro Vice Chancellor, Dr. Manjree Pandit, for her constant encouragement and support.

We extend our heartfelt thanks to the Head of the Department, Dr. Dheeraj K. Dixit, for his valuable guidance and cooperation throughout the course of this work.

We would like to express our deep sense of gratitude to our project supervisor, Dr. Dheeraj Kumar Dixit, for his continuous guidance, constructive suggestions, and unwavering support, which greatly contributed to the successful completion of this project.

We are also thankful to the project coordinators, Dr. Kuldeep K. Tiwari and Dr. Monika Dandotiya, to all faculty members and to our peers for their assistance and encouragement.

Finally, we acknowledge our families for their constant support and motivation and we extend our deepest appreciation and thanks to our friends for their help, encouragement and understanding throughout this semester.

Rudransh Shukla
Enrolment No.: BTMC25O1119
B.Tech. Mathematics and Computing

Vaibhav Sharma
Enrolment No.: BTMC25O1144
B.Tech. Mathematics and Computing

<<Enrolment No.>>

CONTENT

Table of Contents

Declaration by the Candidate i
Abstract ii
Acknowledgement iii
Content iv
Acronyms v
Nomenclature vi
List of Figures vii
List of Tables viii
Chapter 1: Introduction 1
Chapter 2: Literature Survey 3
Chapter 3: Probability and Statistics Foundations 5
Chapter 4: Stochastic Processes and the Stock-Price Model 10
Chapter 5: Financial Mathematics and Option Pricing 14
Chapter 6: Monte Carlo Simulation Theory 18
Chapter 7: Implementation in Python 26
Chapter 8: Results and Discussion 31
Chapter 9: Conclusion 37
Chapter 10: Future Scope 38
References 39

ACRONYMS

Acronym	Full form
AV	Antithetic Variates
BS	Black–Scholes
CDF	Cumulative Distribution Function
CI	Confidence Interval
CLT	Central Limit Theorem
GBM	Geometric Brownian Motion
ITM	In The Money
LCG	Linear Congruential Generator
LLN	Law of Large Numbers
MC	Monte Carlo
OTM	Out of The Money
PDF	Probability Density Function
PRNG	Pseudo-Random Number Generator
SDE	Stochastic Differential Equation
SE	Standard Error

NOMENCLATURE

Symbol	Meaning
S_0	Stock price at time 0
S_t, S_T	Stock price at time t and at expiry T
K	Strike price
T	Time to expiry (years)
r	Continuously compounded risk-free interest rate
σ	Volatility of the stock
μ	Drift (expected return) of the stock
W_t	Standard Brownian motion
Z	Standard normal random variable
$\Phi(\cdot)$	Standard normal cumulative distribution function
d_1, d_2	Black–Scholes parameters
C, P	Price of a European call and put
N	Number of simulated paths
$M, \Delta t$	Number of time steps and step size T/M
$\mathrm{E}[\cdot], \mathrm{Var}(\cdot)$	Expectation and variance
$\hat{C}$	Monte Carlo estimator of the call price
SE	Standard error of the estimate

LIST OF FIGURES

Page No.

Figure 3.1: Central Limit Theorem: sample means of exponential variables follow a normal curve. 9
Figure 4.1: A simple random walk (left) and sample paths of Brownian motion (right). 11
Figure 5.1: Payoff diagrams of a European call (left) and put (right) with strike K = 100. 15
Figure 6.1: Histogram of LCG output (left) and of Box–Muller normals with the N(0,1) curve (right). 21
Figure 7.1: Twenty simulated GBM stock-price paths (S0 = 100, r = 5%, σ = 20%, T = 1 year). 23
Figure 7.2: Histograms of the simulated terminal price (lognormal) and of the call and put payoffs (N = 100,000). 30
Figure 8.1: Convergence of the call price estimate (left) and decay of the standard error (right). 31

LIST OF TABLES

Page No.

Table 1.1: Methodology followed in the project 2
Table 2.1: Key references and their role in the project 3
Table 3.1: Discrete distributions used in the project 6
Table 5.1: Moneyness of European options at expiry 15
Table 5.2: Input parameters of the main experiment 17
Table 7.1: Mapping between theoretical concepts and code fragments 26
Table 7.2: Run-time comparison (measured on the development computer) 27
Table 8.1: Call price versus number of paths (Black–Scholes value = 10.4506) 31
Table 8.2: Monte Carlo prices with N = 100,000 paths 32
Table 8.3: Monte Carlo (N = 100,000) versus Black–Scholes across various strikes 32
Table 8.4: Call price for different numbers of time steps (200,000 paths; exact value 10.4506) 33
Table 8.5: Plain versus antithetic Monte Carlo for the call option 33
Table 8.6: Sensitivity of Monte Carlo call price and standard error to volatility and maturity (N = 100,000) 34

CHAPTER 1: INTRODUCTION

1.1 Background

A *financial option* is a contract that gives its holder the right, but not the obligation, to buy or sell an underlying asset (such as a share) at a fixed price on or before a fixed date. Options are used all over the world to protect against price movements (hedging), to take positions on the market (speculation) and to design structured investment products. Because the holder can walk away when exercising is unprofitable, an option has a non-trivial value today, and the central question of this project is: *what is a fair price for this contract?*

A **European option** can be exercised only on its expiry date. This makes it the simplest option to study, because its payoff depends only on the price of the asset at one instant, the expiry date T. The classical answer to the pricing question for European options is the Black–Scholes formula (1973). That formula is a closed-form expression, but it exists only because the model is simple. For more complicated contracts and models there is usually no closed form, and practitioners then rely on numerical techniques, of which **Monte Carlo simulation** is the most flexible.

Monte Carlo simulation estimates a quantity by repeated random sampling. For option pricing, we randomly simulate many possible future prices of the asset, compute what the option would pay in each scenario, average these payoffs and discount the average back to today. This project implements that idea in Python with NumPy for European call and put options and validates the result against the Black–Scholes formula.

1.2 Problem Statement

Given the current stock price S_0, strike price K, risk-free interest rate r, volatility σ and time to expiry T, estimate the fair price of a European call and put option using Monte Carlo simulation, quantify the statistical error of the estimate, and compare it with the analytical Black–Scholes price.

1.3 Objectives

- To understand and summarise the probability, statistics and stochastic-process concepts needed for Monte Carlo pricing.
- To understand the financial mathematics of European options, risk-neutral valuation and the Black–Scholes formula.
- To implement pseudo-random number generation, the Box–Muller transform and Geometric Brownian Motion path simulation in Python.

- To build a vectorised NumPy pricer for European calls and puts and to measure its accuracy using the standard error and 95% confidence intervals.
- To improve the efficiency of the simulation using antithetic variates and to verify convergence at the $1/\sqrt{N}$ rate.

1.4 Scope and Methodology

The project is deliberately limited to concepts that are covered at our current level of study: elementary probability and statistics, basic stochastic processes (random walk, Brownian motion, Geometric Brownian Motion, an intuitive view of Itô's lemma), financial mathematics (continuous compounding, option payoffs, risk-neutral valuation, Black–Scholes) and Python/NumPy programming. Formal proofs are omitted; each result is stated, explained intuitively and then connected to the application. Wherever a mathematical idea is introduced, the corresponding Python fragment is shown next to it, and the fragments are finally combined into the complete program. Only plain (vanilla) European options on a non-dividend-paying stock with constant volatility and interest rate are considered.

Table 1.1: Methodology followed in the project

Stage	What is done	Chapter
1	Study of probability and statistics foundations	3
2	Stochastic processes and the stock-price model (GBM)	4
3	Financial mathematics: options, risk-neutral valuation, Black–Scholes	5
4	Monte Carlo theory: LLN, random numbers, discretisation, accuracy	6
5	Python / NumPy implementation	7
6	Experiments, results and discussion	8

1.5 Tools Used

The program is written in Python 3 using VS Code and executed from PowerShell. NumPy provides fast array operations and random number generation, and Matplotlib is used to draw the figures [12], [13]. All numerical values reported in this document were produced by running the accompanying code; they are not copied from any other source.

1.6 Organisation of the Report

Chapter 2 reviews the literature. Chapters 3 to 6 develop the mathematical, financial and Monte Carlo theory and, at each step, relate it to the application. Chapter 7 describes the Python implementation. Chapter 8 presents and discusses the results. Chapters 9 and 10 give the conclusion and the future scope.

CHAPTER 2: LITERATURE SURVEY

2.1 Option Pricing Theory

The modern theory of option pricing began with Black and Scholes [1] and, independently, Merton [2], who showed that the payoff of an option can be replicated by continuously trading the underlying stock and a risk-free bond. The price is therefore fixed by the absence of arbitrage and does not depend on investors' expected return on the stock. This led to the closed-form Black–Scholes formula for European options and to the principle of risk-neutral valuation, in which the price is the discounted expected payoff when the stock grows at the risk-free rate. Standard textbook treatments are given by Hull [4], Wilmott, Howison and Dewynne [15] and Shreve [7].

2.2 Monte Carlo Methods

The Monte Carlo method was named and formalised by Metropolis and Ulam [10] for problems that are difficult to solve analytically. Boyle [3] was the first to apply it to option pricing, and Boyle, Broadie and Glasserman [19] later surveyed its use for security pricing. Glasserman [5] gives a complete modern account, including random number generation, discretisation of stochastic differential equations, and variance reduction. The strength of the method is that its error decreases like $1/\sqrt{N}$ irrespective of the dimension of the problem, which makes it the method of choice for products depending on several assets or on the whole price path.

2.3 Random Numbers and Simulation of Stochastic Models

Monte Carlo simulation needs a source of random numbers. Lehmer [9] introduced the congruential generators that are still the basis of many generators, and Knuth [17] analysed their quality. Normal random variables can be obtained from uniform random variables by the Box–Muller transform [8]. Modern libraries, including NumPy, use higher-quality generators such as PCG64 [18]. Kloeden and Platen [11] describe numerical schemes, such as the Euler–Maruyama scheme, for simulating stochastic differential equations. Variance reduction by antithetic variates was introduced by Hammersley and Morton [14].

Table 2.1: Key references and their role in the project

Work	Contribution	Used in this project for
Black & Scholes (1973) [1]	Closed-form price of European options	Analytical benchmark
Merton (1973) [2]	Rational option pricing theory	Risk-neutral valuation idea
Boyle (1977) [3]	First Monte Carlo option pricing	Main simulation approach
Box & Muller (1958) [8]	Normal variates from uniform ones	Random variable generation
Lehmer (1951) [9]	Linear congruential generator	Pseudo-random numbers

Work	Contribution	Used in this project for
Kloeden & Platen (1992) [11]	Euler–Maruyama discretisation	Path simulation
Hammersley & Morton (1956) [14]	Antithetic variates	Variance reduction
Glasserman (2004) [5]	Monte Carlo in financial engineering	Estimator, error analysis

2.4 Research Gap and Contribution of this Work

The works above are written for specialists and typically assume measure theory and advanced stochastic calculus. This project does not try to extend them. Its contribution is a self-contained, step-by-step account at undergraduate level that connects every concept (distributions, CLT, Brownian motion, GBM, risk-neutral pricing, LLN, standard error) to a concrete Python fragment, and then validates the final program quantitatively against the Black–Scholes benchmark.

CHAPTER 3: PROBABILITY AND STATISTICS FOUNDATIONS

Monte Carlo pricing is, at heart, an application of probability and statistics. This chapter revisits the essential concepts directly required for our simulation (standard references are Ross [6] and Feller [16]): basic probability, random variables and expectation, discrete distributions (Bernoulli and Binomial), continuous distributions (Uniform, Exponential, and Normal), independence and correlation, and the Central Limit Theorem. Each section states the mathematical idea, explains how it appears in option pricing, and shows a short Python fragment. All fragments in this chapter assume the following setup.

Listing 3.1: *Common setup used by all later fragments*

```
import math
import numpy as np

rng = np.random.default_rng(42)     # NumPy's random number generator
```

3.1 Probability Fundamentals

An experiment has a *sample space* Ω of all possible outcomes, and an *event* is a subset of Ω. The probability $\Pr(A)$ of an event is a number between 0 and 1 satisfying the standard axioms of probability, with $\Pr(\Omega) = 1$ and $\Pr(A^c) = 1 - \Pr(A)$ for the complement event A^c. In our application the experiment is "observe the stock price at expiry", and a typical event is $A = \{S_T > K\}$, meaning that a European call option finishes in the money.

3.2 Random Variables, Expectation and Variance

A *random variable* X assigns a number to each outcome. Its **expected value** (mean) and **variance** describe the centre and the spread of its distribution:

$$E[X] = \sum_x x\ ,p(x) \quad \text{(discrete)}, \qquad E[X] = \int_{-\infty}^{\infty} xf(x)\ ,dx \quad \text{(continuous)} \tag{3.1}$$

$$Var(X) = E[(X - E[X])^2] = E[X^2] - (E[X])^2 \tag{3.2}$$

Two properties are used repeatedly: expectation is linear, $\mathrm{E}[aX + b] = a\mathrm{E}[X] + b$, while variance scales with the square, $\mathrm{Var}(aX + b) = a^2\mathrm{Var}(X)$. The option price itself is an expected value (Chapter 5), so Monte Carlo is simply a numerical method for computing $\mathrm{E}[\cdot]$.

In practice the true mean and variance are unknown, and we estimate them from N independent samples $X_1, \ldots, X_N$ using the **sample mean** and **sample variance**:

$$\overline{X} = \ frac1N \sum_{i=1}^{N} X_i\,, \qquad s^2 = \ frac1N-1 \sum_{i=1}^{N} (X_i - \overline{X})^2 \tag{3.3}$$

Listing 3.2: *Sample mean and sample variance with NumPy*

```
x = rng.normal(loc=5.0, scale=2.0, size=100_000)
print("sample mean     =", round(x.mean(), 4))
print("sample variance =", round(x.var(ddof=1), 4))   # ddof=1 gives N-1 in the denominator
```

```
Output:
sample mean     = 4.9915
sample variance = 4.0297
```

3.3 Discrete Distributions

A discrete random variable takes countable values. The two discrete distributions required for this project are summarised below, where ${}^{n}C_{k} = \frac{n!}{k!\,(n-k)!}$ represents the number of unordered combinations of n items chosen k at a time.

Table 3.1: Discrete distributions used in the project

Distribution	Probability mass function	Mean	Variance
Bernoulli(p)	$\Pr(X=1)=p,\ \Pr(X=0)=1-p$	p	$p(1-p)$
Binomial(n, p)	${}^{n}C_{k}\,p^{k}(1-p)^{n-k}$	np	$np(1-p)$

Application. Whether one simulated path ends in the money is a Bernoulli trial with success probability $p = \Pr(S_T > K)$. The number of in-the-money paths among N independent paths is therefore Binomial(N, p), with standard deviation $\sqrt{Np(1-p)}$. The fragment below simulates terminal stock prices and checks the Bernoulli/Binomial interpretation; the exact value of p is derived in Chapter 5.

Listing 3.3: *Indicator of an in-the-money path (Bernoulli) and its Binomial count*

```
S0, K, r, sigma, T, N = 100.0, 100.0, 0.05, 0.20, 1.0, 100_000
Z = rng.standard_normal(N)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)

itm = (ST > K)                          # Bernoulli indicator for every path
p_hat = itm.mean()                      # fraction of in-the-money paths
se = math.sqrt(p_hat * (1 - p_hat) / N) # standard error of a Binomial proportion
print(f"in-the-money paths: {itm.sum()} out of {N}")
print(f"estimated P(S_T > K) = {p_hat:.4f} +/- {se:.4f}")
```

```
Output:
in-the-money paths: 55931 out of 100000
estimated P(S_T > K) = 0.5593 +/- 0.0016
```

3.4 Continuous Distributions

3.4.1 Uniform distribution

$X \sim \text{Unif}(a, b)$ has density $f(x) = 1/(b-a)$ on $[a, b]$, mean $(a+b)/2$ and variance $(b-a)^2/12$. The standard uniform $U(0,1)$ is the raw material of all random number generation (Chapter 6): every other distribution is produced by transforming uniform numbers.

3.4.2 Exponential distribution

$X \sim \text{Exp}(\lambda)$ has density $f(x) = \lambda e^{-\lambda x}$ for $x \geq 0$, mean $1/\lambda$ and variance $1/\lambda^2$. It models waiting times and has the **memoryless property**: $\Pr(X > s + t | X > s) = \Pr(X > t)$, so the remaining waiting time does not depend on how long we have already waited.

3.4.3 Normal distribution and the standard normal

The Normal distribution $N(\mu, \sigma^2)$ has density

$$f(x) = \ frac1\sigma \ sqrt2\pi\exp(- \ frac(x-\mu)^2 2\sigma^2) \tag{3.4}$$

A linear transformation of a normal variable is again normal. Subtracting the mean and dividing by the standard deviation gives the **z-score** $Z = (X - \mu)/\sigma$, which follows the *standard normal* distribution $N(0,1)$. Its cumulative distribution function is written $\Phi(z) = \Pr(Z \leq z)$. The normal distribution matters here for two reasons: the random shocks that drive the stock price are normal, and the Black–Scholes formula is written in terms of Φ. For instance $\Phi(1.96) \approx 0.975$, which is where the "95%" of a 95% confidence interval comes from.

Listing 3.4: *Normal density, cumulative distribution function and z-score*

```
def norm_pdf(x, mu=0.0, sigma=1.0):
    return math.exp(-(x - mu) ** 2 / (2 * sigma ** 2)) / (sigma * math.sqrt(2 * math.pi))

def norm_cdf(x):                                    # Phi(x) via the error function
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))

print("pdf at 0      :", round(norm_pdf(0.0), 4))
print("Phi(1.96)     :", round(norm_cdf(1.96), 4))
print("P(-1.96<Z<1.96):", round(norm_cdf(1.96) - norm_cdf(-1.96), 4))
z = (130 - 100) / 20                                # z-score of 130 when mu=100, sigma=20
print("z-score of 130:", z)
```

```
Output:
pdf at 0      : 0.3989
Phi(1.96)     : 0.975
P(-1.96<Z<1.96): 0.95
z-score of 130: 1.5
```

3.5 Independence, Covariance and Correlation

Two random variables X and Y are **independent** if knowledge of one provides no information about the other. Formally, their joint distribution factorises, $\Pr(X \le x,\, Y \le y) = \Pr(X \le x)\Pr(Y \le y)$, which implies $\mathrm{E}[XY] = \mathrm{E}[X]\mathrm{E}[Y]$.

The linear association between two variables is measured by their **covariance** and dimensionless **correlation coefficient** $\rho \in [-1,1]$:

$$Cov(X,Y) = E[(X - E[X])(Y - E[Y])] = E[XY] - E[X]\ E[Y] \tag{3.5}$$

$$\rho = frac\ Cov(X,Y)\ sqrt\ Var(X)\ Var(Y) \tag{3.6}$$

The variance of a linear combination depends directly on covariance:

$$Var(X+Y) = Var(X) + Var(Y) + 2\ Cov(X,Y) \tag{3.7}$$

When X and Y are independent, $\mathrm{Cov}(X,Y) = 0$, and the variance of the sum simplifies to $\mathrm{Var}(X) + \mathrm{Var}(Y)$.

Application. (i) The random shocks of different time steps in a simulated price path are independent, so their variances simply add: this property underpins Brownian motion (Chapter 4). (ii) In the antithetic-variates technique (Chapter 6), we intentionally simulate two *negatively correlated* payoff streams Y and Y' ($\rho < 0$). The variance of their sample average is $\mathrm{Var}((Y+Y')/2) = \frac{1}{4}\mathrm{Var}(Y) + \frac{1}{4}\mathrm{Var}(Y') + \frac{1}{2}\mathrm{Cov}(Y,Y')$, which is substantially smaller than $\mathrm{Var}(Y)/2$ because the covariance is negative.

3.6 The Central Limit Theorem

Let $X_1, \ldots, X_N$ be independent and identically distributed with mean μ and finite variance σ^2. The **Central Limit Theorem (CLT)** states that for large N the sample mean is approximately normal, whatever the shape of the original distribution:

$$frac\overline{X}_N - \mu\sigma/\ sqrtN \ \longrightarrow\ N(0,1) \quad \text{as } N \to \infty, \qquad \text{i.e.} \quad \overline{X}_N \approx N(\mu,\ frac\sigma^2 N) \tag{3.8}$$

Application. The Monte Carlo price is the sample mean of N independent discounted payoffs. By the CLT, it is approximately normal with a standard deviation $\sigma/\sqrt{N}$ (the *standard error*). This is the reason why we can attach a 95% confidence interval to the estimate (Chapter 6). Figure 3.1 shows the theorem in action: the individual values come from the strongly skewed exponential distribution, yet the histogram of their sample means is bell-shaped.

Listing 3.5: *Numerical illustration of the Central Limit Theorem*

```
# Sample means of 30 exponential(1) variables, repeated 20,000 times
means = rng.exponential(scale=1.0, size=(20_000, 30)).mean(axis=1)
print("mean of sample means :", round(means.mean(), 4), "(theory 1.0000)")
print("std  of sample means :", round(means.std(), 4), f"(theory {1/math.sqrt(30):.4f})")
```

```
Output:
mean of sample means : 0.9991 (theory 1.0000)
std  of sample means : 0.1805 (theory 0.1826)
```

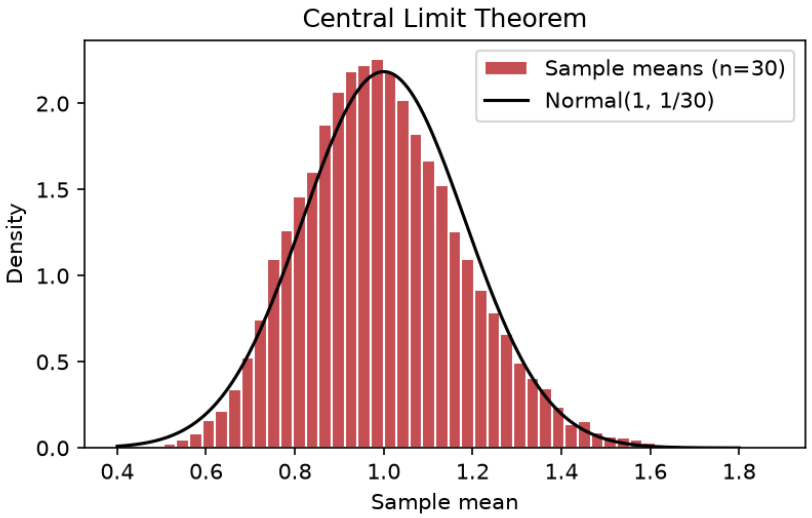


Figure 3.1: Central Limit Theorem: sample means of exponential variables follow a normal curve.

CHAPTER 4: STOCHASTIC PROCESSES AND THE STOCK-PRICE MODEL

A *stochastic process* is a collection of random variables indexed by time. To price an option we need a model for the random evolution of the stock price. This chapter builds that model step by step: random walk, Brownian motion, the lognormal distribution, Geometric Brownian Motion and, finally, an intuitive view of Itô's lemma that yields the exact solution.

4.1 Random Walks and the Markov Property

A stochastic process possesses the **Markov property** if its future evolution depends on the past only through its present state: $\Pr(X_{t+s} \in A | X_u, u \le t) = \Pr(X_{t+s} \in A | X_t)$. In financial modelling, this means that the current stock price summarizes all past historical information, so future price movements require no additional memory of past trajectories. Because Geometric Brownian Motion satisfies the Markov property, simulated asset paths can be generated sequentially one step at a time, where each successive price depends solely on the current price and an independent random shock. This memoryless structure substantially simplifies both path simulation and recursive valuation algorithms.

4.2 Random Walks

A **simple random walk** starts at 0 and, at each step, moves up or down by one unit with probability $rac12$ each: $S_n = \xi_1 + \xi_2 + \cdots + \xi_n$ where $\xi_i = \pm 1$. Then $\mathrm{E}[S_n] = 0$ and $\mathrm{Var}(S_n) = n$, so the typical distance from the start grows like $\sqrt{n}$, not like n. The walk is a Markov process with independent increments.

Listing 4.1: *Simulating a random walk*

```
steps = rng.choice([-1, 1], size=100)          # xi_i = +1 or -1 with probability 1/2
walk = np.cumsum(steps)                        # S_n = xi_1 + ... + xi_n
print("final position after 100 steps:", walk[-1])
ends = rng.choice([-1, 1], size=(20_000, 100)).sum(axis=1)
print("variance of S_100 over 20,000 walks:", round(ends.var(), 2), "(theory 100)")
```

```
Output:
final position after 100 steps: 6
variance of S_100 over 20,000 walks: 99.8 (theory 100)
```

4.3 Standard Brownian Motion (Wiener Process)

If we shrink the step size to $\sqrt{\Delta t}$ and the time between steps to Δt, and let $\Delta t \to 0$, the random walk converges to **standard Brownian motion** W_t. It is defined by four properties:

- $W_0 = 0$;
- independent increments: $W_t - W_s$ is independent of the past for $s < t$;

- normally distributed increments: $W_t - W_s \sim N(0,\, t-s)$;
- continuous paths (which are, however, extremely jagged and nowhere differentiable).

Consequently an increment over a short time Δt can be generated as $\Delta W = \sqrt{\Delta t}\, Z$ with $Z \sim N(0,1)$, and $\mathrm{Var}(W_t) = t$. A key fact for Section 4.5 is that $(\Delta W)^2 \approx \Delta t$: the squared increment is not negligible, unlike in ordinary calculus.

Listing 4.2: *Brownian motion path from independent normal increments*

```
T_, n = 1.0, 1000
dt = T_ / n
dW = math.sqrt(dt) * rng.standard_normal(n)    # independent N(0, dt) increments
W = np.concatenate(([0.0], np.cumsum(dW)))     # W_0 = 0, W_t = sum of increments
print("W_T for this path:", round(W[-1], 4))
WT = math.sqrt(T_) * rng.standard_normal(100_000)
print("Var(W_T) over many paths:", round(WT.var(), 4), "(theory T = 1)")
```

```
Output:
W_T for this path: -1.3204
Var(W_T) over many paths: 1.0001 (theory T = 1)
```

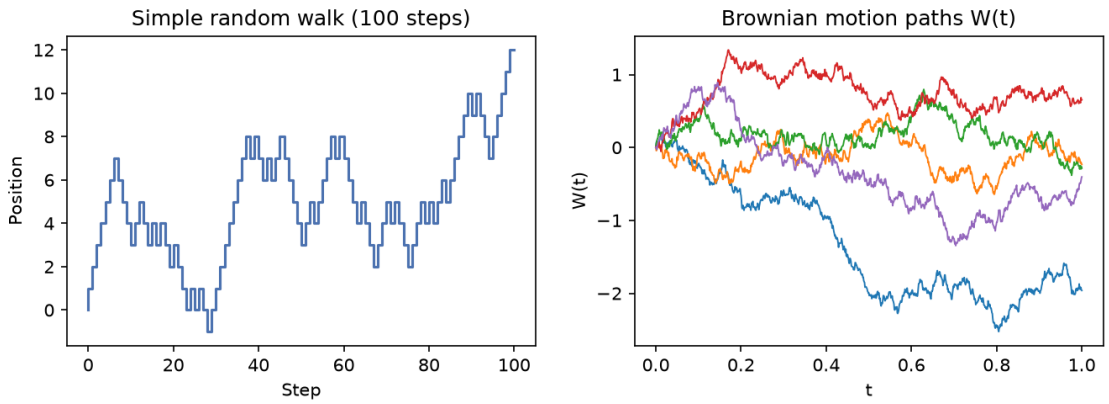


Figure 4.1: A simple random walk (left) and sample paths of Brownian motion (right).

4.4 The Lognormal Distribution

A positive random variable Y is **lognormal** if $\ln Y$ is normal. If $X \sim N(\mu, \sigma^2)$ and $Y = e^X$ then

$$f_Y(y) = \ frac1y\sigma\ sqrt2\pi\exp(-\ frac(\ln y-\mu)^2 2\sigma^2)\ \ (y > 0), \qquad E[Y] = e^{\mu+\sigma^2/2}, \qquad \mathrm{median}(Y) = e^{\mu} \tag{4.1}$$

Why lognormal for stock prices? Prices grow *multiplicatively*: $S_n = S_0 \prod\ (1+R_i)$, where R_i are the periodic returns. Taking logarithms turns the product into a sum, $\ln S_n = \ln S_0 + \sum\ \ln(1+R_i)$, and by the CLT such a sum is approximately normal. Hence the price itself is lognormal: it can never become negative (a share holder's loss is limited to the amount

invested), but it has no upper bound and its distribution is skewed to the right. The mean of a lognormal variable exceeds its median because of the long right tail.

Listing 4.3: *Lognormal samples: positivity and mean versus median*

```
mu, sig = 0.0, 0.5
Y = np.exp(rng.normal(mu, sig, 1_000_000))     # lognormal sample: exp of a normal
print("min of sample          :", round(Y.min(), 4), "(always > 0)")
print("sample mean            :", round(Y.mean(), 4), " theory exp(mu + sig^2/2) =",
round(math.exp(mu + sig**2 / 2), 4))
print("sample median          :", round(float(np.median(Y)), 4), " theory exp(mu) =",
round(math.exp(mu), 4))
```

```
Output:
min of sample          : 0.0803 (always > 0)
sample mean            : 1.1334  theory exp(mu + sig^2/2) = 1.1331
sample median          : 1.0003  theory exp(mu) = 1.0
```

4.5 Geometric Brownian Motion (GBM)

The standard model for a stock price assumes that the *relative* change in price over a short time has a predictable part and a random part. In differential form, the price S_t follows the stochastic differential equation

$$dS_t = \mu \ , S_t \ , dt + \sigma \ , S_t \ , dW_t \tag{4.2}$$

where μ is the drift (expected rate of return), σ the volatility (standard deviation of returns per unit time) and W_t a Brownian motion. Dividing by S_t shows the meaning: the return dS/S equals $\mu\, dt$ plus a random shock of size $\sigma\, dW$. Because the random term is proportional to S_t, a ₹1 move is far more likely for a share at ₹1000 than at ₹10, exactly as in real markets, and the price cannot become negative. In discrete time steps of length Δt:

$$frac\Delta SS = \mu \ , \Delta t + \sigma \ \ sqrt\Delta t \ \ , Z, \qquad Z \sim N(0,1) \tag{4.3}$$

4.6 Itô's Lemma: the Intuition

In ordinary calculus, if x changes by a small amount dx, the term $(dx)^2$ is negligible and the chain rule has only a first-order term. For a Brownian motion, however, $(dW)^2 = dt$, which is of the same order as dt itself and therefore cannot be ignored. **Itô's lemma** is the chain rule that keeps this extra term. For a smooth function $f(S,t)$ of a stock price following GBM it reads

$$df = (\ frac\partial f\partial t + \mu S \ \ frac\partial f\partial S + \ \ frac12\sigma^2S^2 \ \ frac\partial^2 f\partial S^2)dt + \sigma S \ \ frac\partial f\partial S \ \ , dW \tag{4.4}$$

Applying it to $f = \ln S$, with $\partial f/\partial S = 1/S$ and $\partial^2 f/\partial S^2 = -1/S^2$, we obtain a very simple equation:

$$d(\ln S_t) = (\mu - \ frac\sigma^2 2)dt + \sigma \ , dW_t \tag{4.5}$$

whose right-hand side has constant coefficients, so it can be integrated directly from 0 to T using $W_T = \sqrt{T}\, Z$:

$$S_T = S_0 \exp\big((\mu - \ frac\sigma^2 2)T + \sigma \ sqrtT \ , Z\big), \qquad Z \sim N(0,1) \tag{4.6}$$

Thus $\ln S_T \sim N(\ln S_0 + (\mu - \sigma^2/2)T,\ \sigma^2 T)$ and S_T is lognormal, with $\mathrm{E}[S_T] = S_0 e^{\mu T}$. The correction $-\sigma^2/2$ is the Itô term: it reflects that, with volatile multiplicative returns, the *typical* (median) growth rate is lower than the *average* growth rate μ. The fragment below checks these statements numerically (here $\mu = 8$).

Listing 4.4: *Terminal price from the exact GBM solution*

```
S0, mu, sigma, T = 100.0, 0.08, 0.20, 1.0
Z = rng.standard_normal(1_000_000)
ST = S0 * np.exp((mu - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)   # exact GBM solution

print("E[S_T]  simulated:", round(ST.mean(), 3), "  theory S0*exp(mu*T) =", round(S0 *
math.exp(mu * T), 3))
print("mean of ln S_T    :", round(np.log(ST).mean(), 4), "  theory", round(math.log(S0) + (mu
- 0.5 * sigma**2) * T, 4))
print("std  of ln S_T    :", round(np.log(ST).std(), 4), "  theory", sigma * math.sqrt(T))
```

```
Output:
E[S_T]  simulated: 108.345   theory S0*exp(mu*T) = 108.329
mean of ln S_T    : 4.6653   theory 4.6652
std  of ln S_T    : 0.2   theory 0.2
```

CHAPTER 5: FINANCIAL MATHEMATICS AND OPTION PRICING

This chapter introduces the financial side of the project: how money grows over time, what a European option is, why its price can be computed as a discounted expected payoff, and what the Black–Scholes formula says. Each concept is followed by a Python fragment.

5.1 Time Value of Money and Continuous Compounding

Money available today is worth more than the same amount in the future because it can earn interest. If an amount PV is invested at an annual rate r compounded m times per year for T years, it grows to

$$FV = PV\left(1+\frac{r}{m}\right)^{mT} \quad \longrightarrow \quad FV = PV\,e^{rT} \quad \text{as } m \to \infty \tag{5.1}$$

Letting the compounding frequency m grow without bound gives **continuous compounding**, in which $FV = PV\,e^{rT}$. Reversing the argument, a payment FV due at time T has present value $PV = FV\,e^{-rT}$; the factor e^{-rT} is the *discount factor*. Continuous compounding is used in option pricing because it combines neatly with the exponential form of the GBM solution.

Listing 5.1: *Growth under different compounding frequencies*

```
PV, r, T = 100.0, 0.05, 1.0
for m in (1, 12, 365):
    print(f"compounded {m:>3} times a year: FV = {PV * (1 + r / m) ** (m * T):.4f}")
print(f"continuous compounding     : FV = {PV * math.exp(r * T):.4f}")
print(f"discount factor exp(-rT)   : {math.exp(-r * T):.4f}")
```

```
Output:
compounded   1 times a year: FV = 105.0000
compounded  12 times a year: FV = 105.1162
compounded 365 times a year: FV = 105.1267
continuous compounding     : FV = 105.1271
discount factor exp(-rT)   : 0.9512
```

5.2 European Options: Mechanics and Payoffs

A **European call option** gives the holder the right to *buy* one unit of the underlying asset at the **strike price** K on the **expiration date** T; a **European put option** gives the right to *sell* at K on T. The buyer (holder) pays a **premium** today, which is the price we want to compute; the seller (writer) takes on the obligation. “European” means exercise is allowed only at T (American options may be exercised at any time up to T).

At expiry the holder exercises only if it is profitable. For a call, buying at K is worthwhile when the market price S_T is higher than K, and the gain is $S_T - K$; otherwise the call expires worthless. Similarly for a put. The **payoff functions** are therefore

$$\text{Call payoff} = \max(S_T - K,\ 0), \qquad \text{Put payoff} = \max(K - S_T,\ 0) \tag{5.2}$$

Table 5.1: Moneyness of European options at expiry

Situation at expiry	Call	Put
$S_T > K$	in the money (pays $S_T - K$)	out of the money (pays 0)
$S_T = K$	at the money (pays 0)	at the money (pays 0)
$S_T < K$	out of the money (pays 0)	in the money (pays $K - S_T$)

Listing 5.2: *Payoff functions applied to an array of terminal prices*

```
def payoffs(S_T, K, option="call"):
    if option == "call":
        return np.maximum(S_T - K, 0.0)        # max(S_T - K, 0), element by element
    return np.maximum(K - S_T, 0.0)            # max(K - S_T, 0)

S_T = np.array([80.0, 100.0, 120.0])
print("S_T  :", S_T)
print("call :", payoffs(S_T, 100.0, "call"))
print("put  :", payoffs(S_T, 100.0, "put"))
```

```
Output:
S_T  : [ 80. 100. 120.]
call : [ 0.  0. 20.]
put  : [20.  0.  0.]
```

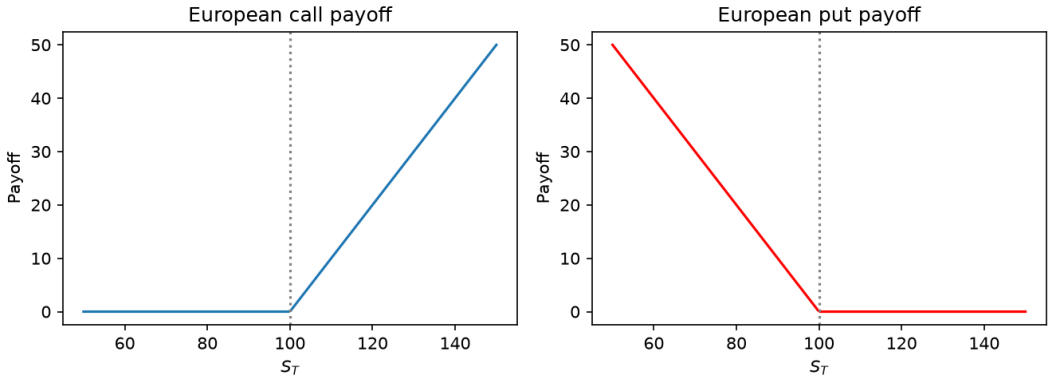


Figure 5.1: Payoff diagrams of a European call (left) and put (right) with strike K = 100.

5.3 Risk-Neutral Valuation

In Chapter 4 the stock drifts at the expected return μ, a number that depends on investors' risk appetite and is difficult to estimate. Remarkably, the option price does **not** depend on μ. The reason is *no-arbitrage*: an option can be hedged by trading the stock, so its price must equal

the cost of the hedging portfolio, and that cost involves only σ and the risk-free rate r — not μ. A convenient way to compute it is to pretend that investors are risk-neutral, i.e. that every asset earns just the risk-free rate. The **risk-neutral valuation principle** states

$$C = e^{-rT}\,\mathrm{E}^{Q}[\max(S_T - K, 0)], \qquad P = e^{-rT}\,\mathrm{E}^{Q}[\max(K - S_T, 0)] \tag{5.3}$$

where E^{Q} is the expectation in the risk-neutral world, in which the stock follows GBM with drift r instead of μ:

$$S_T = S_0 \exp\left(\left(r - \frac{\sigma^2}{2}\right)T + \sigma\ \ sqrtT\ Z\right), \qquad Z \sim N(0,1) \tag{5.4}$$

Why discount at the risk-free rate? Under the risk-neutral measure all risk is "priced in", so the expected payoff is not further adjusted for risk; it only has to be moved back in time from T to today, which is done with the risk-free discount factor e^{-rT}. A consistency check is that the discounted expected stock price must equal today's price, $e^{-rT}\mathrm{E}^{Q}[S_T] = S_0$. The fragment below verifies this numerically.

Listing 5.3: *Check that the discounted risk-neutral stock price averages to S0*

```
S0, K, r, sigma, T, N = 100.0, 100.0, 0.05, 0.20, 1.0, 1_000_000
Z = rng.standard_normal(N)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)   # risk-neutral: drift
r
print("e^(-rT) * mean(S_T) =", round(math.exp(-r * T) * ST.mean(), 3), "(should be close to S0
= 100)")
```

```
Output:
e^(-rT) * mean(S_T) = 100.012 (should be close to S0 = 100)
```

5.4 The Black–Scholes Formula

Under the GBM model with constant r and σ, no dividends, and frictionless markets, the expectations above can be evaluated exactly. The result is the **Black–Scholes formula**:

$$C = S_0\,\Phi(d_1) - Ke^{-rT}\,\Phi(d_2), \qquad P = Ke^{-rT}\,\Phi(-d_2) - S_0\,\Phi(-d_1) \tag{5.5}$$

$$d_1 = \frac{\ln(S_0/K) + (r + \sigma^2/2)T}{\sigma\ \ sqrtT}, \qquad d_2 = d_1 - \sigma\ \ sqrtT \tag{5.6}$$

where Φ is the standard normal cumulative distribution function. The terms have a clear meaning: $\Phi(d_2)$ is the risk-neutral probability that the option finishes in the money, $Ke^{-rT}\Phi(d_2)$ is the discounted expected amount paid for the stock, and $S_0\Phi(d_1)$ is the

corresponding discounted expected value of the stock received. Calls and puts are linked by **put–call parity**:

$$C - P = S_0 - Ke^{-rT} \tag{5.7}$$

The Black–Scholes formula is the exact answer to the problem that our simulation solves approximately. This makes it an ideal *benchmark*: any systematic difference between the Monte Carlo and Black–Scholes prices would indicate an error in the code.

Listing 5.4: *Black–Scholes benchmark*

```
def norm_cdf(x):
    """Standard normal CDF Phi(x) using the error function."""
    return 0.5 * (1.0 + math.erf(x / math.sqrt(2.0)))



def black_scholes(S0, K, r, sigma, T, option="call"):
    d1 = (math.log(S0 / K) + (r + 0.5 * sigma ** 2) * T) / (sigma * math.sqrt(T))
    d2 = d1 - sigma * math.sqrt(T)
    if option == "call":
        return S0 * norm_cdf(d1) - K * math.exp(-r * T) * norm_cdf(d2)
    return K * math.exp(-r * T) * norm_cdf(-d2) - S0 * norm_cdf(-d1)

S0, K, r, sigma, T = 100.0, 100.0, 0.05, 0.20, 1.0
C = black_scholes(S0, K, r, sigma, T, "call")
P = black_scholes(S0, K, r, sigma, T, "put")
print(f"Black-Scholes call = {C:.4f}")
print(f"Black-Scholes put  = {P:.4f}")
print(f"C - P = {C - P:.4f},   S0 - K*exp(-rT) = {S0 - K * math.exp(-r * T):.4f}")
d2 = (math.log(S0 / K) + (r - 0.5 * sigma**2) * T) / (sigma * math.sqrt(T))
print(f"risk-neutral P(S_T > K) = Phi(d2) = {norm_cdf(d2):.4f}")
```

```
Output:
Black-Scholes call = 10.4506
Black-Scholes put  = 5.5735
C - P = 4.8771,   S0 - K*exp(-rT) = 4.8771
risk-neutral P(S_T > K) = Phi(d2) = 0.5596
```

5.5 Parameters Used in the Project

Table 5.2: Input parameters of the main experiment

Symbol	Meaning	Value
S_0	Initial stock price	100
K	Strike price	100
r	Risk-free interest rate (continuous)	5% per year
σ	Volatility	20% per year
T	Time to expiry	1 year
N	Number of simulated paths (main experiment)	100,000
seed	Random seed (for reproducibility)	42

CHAPTER 6: MONTE CARLO SIMULATION THEORY

Having a stock-price model (Chapter 4) and a pricing principle (Chapter 5), this chapter explains how the expectation in the pricing formula is computed by simulation. The concepts are: the Monte Carlo estimator, the Law of Large Numbers, pseudo-random number generation, generation of normal random variables, discretisation of GBM, error analysis, and variance reduction.

6.1 The Monte Carlo Method

Suppose we want $\theta = \mathrm{E}[g(X)]$ but cannot compute the integral analytically. If we can draw independent samples $X^{(1)}, \dots, X^{(N)}$ from the distribution of X, the average of g over the samples is a natural estimate of θ. For a European call, g is the discounted payoff and the samples are simulated terminal prices:

$$\hat{C}_N = e^{-rT}\,\frac{1}{N}\sum_{i=1}^{N}\max\left(S_T^{(i)} - K,\,0\right) \tag{6.1}$$

The algorithm therefore consists of the following steps, each of which becomes a short Python fragment in Chapter 7:

1. generate N independent standard normal random numbers $Z^{(1)}, \dots, Z^{(N)}$;
2. turn each into a terminal stock price $S_T^{(i)} = S_0\exp((r - \sigma^2/2)T + \sigma\sqrt{T}Z^{(i)})$;
3. evaluate the payoff $\max(S_T^{(i)} - K, 0)$ (call) or $\max(K - S_T^{(i)}, 0)$ (put) for every path;
4. average the payoffs and multiply by the discount factor e^{-rT} to obtain the price estimate;
5. compute the standard error and the 95% confidence interval to state how accurate the estimate is.

6.2 The Law of Large Numbers

The **Law of Large Numbers (LLN)** says that the average of independent, identically distributed samples converges to the true expected value as the number of samples grows:

$$\overline{X}_N = \frac{1}{N}\sum_{i=1}^{N} X_i \;\longrightarrow\; \mathrm{E}[X] \quad \text{as } N \to \infty \tag{6.2}$$

This is the theoretical justification for Monte Carlo pricing: *averaging thousands of simulated payoffs finds the true price*. The LLN guarantees convergence, but does not say how fast; that is the role of the CLT and the standard error (Section 6.6). The fragment below prints the running estimate of the call price for increasing N and compares it with the Black–Scholes value (the corresponding graph is Figure 8.1).

Listing 6.1: *The estimate approaches the true price as N grows (LLN)*

```
from mc_option_pricing import black_scholes
S0, K, r, sigma, T = 100.0, 100.0, 0.05, 0.20, 1.0
rng = np.random.default_rng(11)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) *
rng.standard_normal(1_000_000))
disc_payoff = math.exp(-r * T) * np.maximum(ST - K, 0.0)
running_mean = np.cumsum(disc_payoff) / np.arange(1, len(disc_payoff) + 1)   # estimate after
n paths
for n in (10, 100, 1_000, 10_000, 100_000, 1_000_000):
    print(f"N = {n:>9,}   estimate = {running_mean[n - 1]:.4f}")
print(f"Black-Scholes value     = {black_scholes(S0, K, r, sigma, T, 'call'):.4f}")
```

```
Output:
N =        10   estimate = 10.2422
N =       100   estimate = 10.1141
N =     1,000   estimate = 10.7168
N =    10,000   estimate = 10.6368
N =   100,000   estimate = 10.4534
N = 1,000,000   estimate = 10.4491
Black-Scholes value     = 10.4506
```

6.3 Pseudo-Random Number Generators and the LCG

A computer cannot produce truly random numbers; it produces *pseudo-random* numbers by a deterministic recurrence that looks random. The classical example is the **Linear Congruential Generator (LCG)** [9]:

$$X_{n+1} = (a\,X_n + c) \bmod m, \qquad U_n = \frac{X_n}{m} \in [0,1) \tag{6.3}$$

Starting from a *seed* X_0, each state is computed from the previous one using the multiplier a, the increment c and the modulus m; dividing by m gives numbers that behave like independent $U(0,1)$ samples. Since there are only m possible states, the sequence repeats after at most m steps (its *period*); a full period is obtained when c and m have no common factor, $a-1$ is divisible by every prime factor of m, and by 4 if m is divisible by 4. A tiny example makes the mechanism visible: with $a = 5$, $c = 3$, $m = 16$ and seed 7 the states are $7 \to 6 \to 1 \to 8 \to 11 \to 10 \to \cdots$ (for instance $5 \cdot 7 + 3 = 38$ and $38 \bmod 16 = 6$). The class below implements the same recurrence with larger constants (those used in the classical C library), and the quality of its output is checked against the exact moments of $U(0,1)$: mean $1/2$ and variance $1/12 \approx 0.0833$.

Listing 6.2: *Linear congruential generator (LCG)*

```
class LCG:
    """Linear Congruential Generator: X_{n+1} = (a X_n + c) mod m."""

    def __init__(self, seed=12345, a=1103515245, c=12345, m=2**31):
        self.state, self.a, self.c, self.m = seed, a, c, m

    def uniform(self, n):
        """Return n pseudo-random numbers in (0, 1)."""
        out = np.empty(n)
        for i in range(n):
            self.state = (self.a * self.state + self.c) % self.m
            out[i] = (self.state + 0.5) / self.m   # +0.5 keeps values strictly in (0,1)
        return out

toy = mc.LCG(seed=7, a=5, c=3, m=16)
states = []
for _ in range(6):
    toy.state = (toy.a * toy.state + toy.c) % toy.m
    states.append(toy.state)
print("toy LCG states:", states)

g = mc.LCG(seed=2026)
u = g.uniform(20_000)
print("first five U(0,1):", np.round(u[:5], 6))
print(f"mean = {u.mean():.4f} (theory 0.5000),  variance = {u.var():.4f} (theory {1/12:.4f})")
```

```
Output:
toy LCG states: [6, 1, 8, 11, 10, 5]
first five U(0,1): [0.089137 0.364932 0.078967 0.170371 0.687997]
mean = 0.4987 (theory 0.5000),  variance = 0.0835 (theory 0.0833)
```

LCGs are simple and fast but have known weaknesses (for example, points in higher dimensions fall on a small number of hyperplanes, and low-order bits have short periods). Professional libraries therefore use better generators. NumPy's `default_rng` is based on PCG64 [18], and this is the generator used for the large experiments of this project, while the LCG above is used to demonstrate, from scratch, how a complete pricer can be built (Section 7.6).

6.4 Generating Random Variables

6.4.1 Inverse transform method

If F is the cumulative distribution function of a random variable X and $U \sim U(0,1)$, then $F^{-1}(U)$ has distribution F. For instance, for the exponential distribution, $F(x) = 1 - e^{-\lambda x}$, so inverting gives $X = -\ln(1-U)/\lambda$. The method works whenever F^{-1} is available in closed form.

Listing 6.3: *Inverse transform method for the exponential distribution*

```
lam = 2.0
U = rng.random(1_000_000)                # U(0,1) samples
X = -np.log(1.0 - U) / lam               # inverse transform for Exp(lambda)
print("sample mean:", round(X.mean(), 4), "(theory 1/lambda =", 1 / lam, ")")
```

```
Output:
sample mean: 0.5001 (theory 1/lambda = 0.5 )
```

6.4.2 Box–Muller transform

The normal cumulative distribution function has no closed-form inverse, so a different idea is used. If U_1 and U_2 are independent $U(0,1)$ variables, then the **Box–Muller transform** [8]

$$Z_1 = \ sqrt - 2\ln U_1 \cos(2\pi U_2), \qquad Z_2 = \ sqrt - 2\ln U_1 \sin(2\pi U_2) \tag{6.4}$$

produces two *independent* standard normal variables. The intuition is polar coordinates: $-2\ln U_1$ gives the squared radius of a two-dimensional standard normal point, and $2\pi U_2$ is its angle, which is uniform.

Listing 6.4: *Box–Muller transform applied to LCG uniforms*

```
def box_muller(u1, u2):
    """Turn two independent U(0,1) arrays into two independent N(0,1) arrays."""
    r = np.sqrt(-2.0 * np.log(u1))
    theta = 2.0 * np.pi * u2
    return r * np.cos(theta), r * np.sin(theta)


u = mc.LCG(seed=2026).uniform(20_000)           # uniforms from our own LCG
z1, z2 = box_muller(u[:10_000], u[10_000:])
print(f"Z1: mean = {z1.mean():.4f}, std = {z1.std():.4f}   (theory 0 and 1)")
print(f"correlation between Z1 and Z2 = {np.corrcoef(z1, z2)[0, 1]:.4f}  (theory 0)")
```

```
Output:
Z1: mean = 0.0144, std = 1.0006   (theory 0 and 1)
correlation between Z1 and Z2 = 0.0043  (theory 0)
```

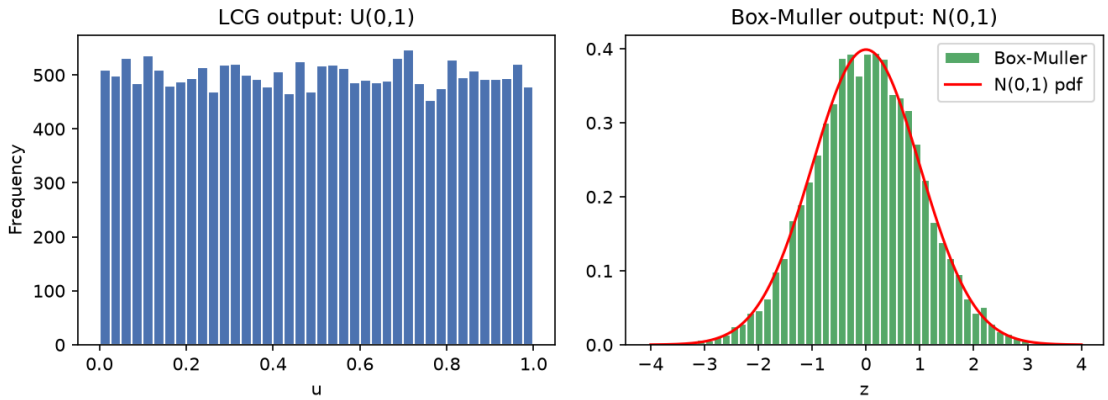


Figure 6.1: Histogram of LCG output (left) and of Box–Muller normals with the N(0,1) curve (right).

6.5 Discretising Geometric Brownian Motion

For a European option only the terminal price matters, and the exact solution of Chapter 4 allows us to jump from 0 to T in one step. For plotting price paths, and for path-dependent options (Chapter 10), we need prices at intermediate times $t_i = i\Delta t$ with $\Delta t = T/M$. The **Euler–Maruyama** scheme [11] replaces the differentials of the SDE by small finite steps:

$$\begin{aligned} S_{i+1} &= S_i + r\,S_i\,\Delta t + \sigma\,S_i\ \ sqrt\Delta t\,Z_i \\ &= S_i(1 + r\,\Delta t + \sigma\ \ sqrt\Delta t\,Z_i), \qquad Z_i \sim N(0,1) \end{aligned} \tag{6.5}$$

This is the discrete return model of Section 4.4 with the risk-neutral drift r. It is only approximate, with an error that shrinks as $\Delta t \to 0$. Applying the exact solution on each sub-interval instead gives the **log scheme**, which has no discretisation error for GBM:

$$S_{i+1} = S_i \exp\left(\left(r - \frac{\sigma^2}{2}\right)\Delta t + \sigma\ \ sqrt\Delta t\,Z_i\right) \tag{6.6}$$

Both schemes are implemented with NumPy without any Python loop over paths or steps: all random numbers are drawn at once into an array of shape (paths × steps), and the recursive product (or sum of logarithms) is evaluated with `cumprod` (or `cumsum`) along the time axis.

Listing 6.5: *Path generator for Geometric Brownian Motion*

```
def simulate_gbm_paths(S0, r, sigma, T, n_steps, n_paths, rng, scheme="euler"):
    """Return array (n_paths, n_steps+1) of simulated stock paths.

    scheme = "euler": S_{t+dt} = S_t (1 + r dt + sigma sqrt(dt) Z)   (Euler-Maruyama)
    scheme = "log"  : S_{t+dt} = S_t exp((r - sigma^2/2) dt + sigma sqrt(dt) Z)
    """
    dt = T / n_steps
    Z = rng.standard_normal((n_paths, n_steps))           # all random numbers at once
    paths = np.empty((n_paths, n_steps + 1))
    paths[:, 0] = S0
    if scheme == "euler":
        growth = 1.0 + r * dt + sigma * math.sqrt(dt) * Z
        paths[:, 1:] = S0 * np.cumprod(growth, axis=1)
    else:
        log_inc = (r - 0.5 * sigma ** 2) * dt + sigma * math.sqrt(dt) * Z
        paths[:, 1:] = S0 * np.exp(np.cumsum(log_inc, axis=1))
    return paths

rng = np.random.default_rng(3)
paths = simulate_gbm_paths(100.0, 0.05, 0.20, 1.0, n_steps=252, n_paths=5, rng=rng,
scheme="log")
print("array shape (paths, time points):", paths.shape)
print("first path, first 5 prices      :", np.round(paths[0, :5], 3))
```

```
Output:
array shape (paths, time points): (5, 253)
first path, first 5 prices      : [100.    102.617  99.377  99.914  99.214]
```

Figure 7.1 shows 20 simulated paths. The effect of the time step on the price is examined in Chapter 8.

6.6 Estimator, Standard Error and Confidence Interval

Each Monte Carlo estimate is itself random: running the simulation again with different random numbers gives a slightly different price. To quantify this uncertainty, let $Y_i = e^{-rT}\max(S_T^{(i)} - K, 0)$ be the discounted payoff of path i. The estimate is $\hat{C} = \overline{Y}$, the sample mean. By the CLT, $\hat{C}$ is approximately normal with mean C and standard deviation $\sigma_Y/\sqrt{N}$, where σ_Y is the standard deviation of the individual Y_i. Replacing σ_Y by the sample standard deviation s gives the **standard error** and a **95% confidence interval**:

$$\mathrm{SE} = \frac{s}{sqrtN}, \qquad 95\%\ \mathrm{CI} = \overline{Y} \pm 1.96\ \mathrm{SE} \tag{6.7}$$

The factor 1.96 is the value z with $\Phi(z) = 0.975$. The interval means: if the whole experiment were repeated many times, about 95% of the intervals so obtained would contain the true price. Two practical consequences follow. First, the error decreases like $1/\sqrt{N}$: to make the estimate ten times more accurate we need a hundred times more paths. Second, a Monte Carlo price must always be reported together with its standard error.

Listing 6.6: *Price, standard error and 95% confidence interval*

```
def summarise(discounted):
    """Mean, standard error and 95% confidence interval of discounted payoffs."""
    n = len(discounted)
    price = discounted.mean()
    std_err = discounted.std(ddof=1) / math.sqrt(n)
    return price, std_err, (price - 1.96 * std_err, price + 1.96 * std_err)


S0, K, r, sigma, T = 100.0, 100.0, 0.05, 0.20, 1.0
rng = np.random.default_rng(42)
ST = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) *
rng.standard_normal(100_000))
disc = math.exp(-r * T) * np.maximum(ST - K, 0.0)          # discounted payoff of every path
price, se, (lo, hi) = summarise(disc)
print(f"price = {price:.4f}   standard error = {se:.4f}")
print(f"95% confidence interval = ({lo:.4f}, {hi:.4f})")
```

```
Output:
price = 10.4205   standard error = 0.0468
95% confidence interval = (10.3289, 10.5122)
```

6.7 Variance Reduction: Antithetic Variates

Since the standard error is $s/\sqrt{N}$, accuracy can also be improved by reducing s rather than increasing N. The **antithetic variates** technique [14] exploits the symmetry of the normal distribution: if Z is a valid standard normal draw, then so is $-Z$. For each draw we compute

two payoffs, from Z and from $-Z$, and use their average. Denoting the two discounted payoffs by Y and Y', the variance of the average is

$$\mathrm{Var}\left(\frac{Y+Y'}{2}\right) = \frac{\mathrm{Var}(Y) + \mathrm{Var}(Y') + 2\mathrm{Cov}(Y,Y')}{4} = \frac{\mathrm{Var}(Y)}{2}\,(1+\rho) \qquad (6.8)$$

where ρ is the correlation between Y and Y'. The payoff of a call is an increasing function of Z, so Y and Y' are negatively correlated ($\rho < 0$) and the variance falls below the plain-sampling value $\mathrm{Var}(Y)/2$ per pair. The fragment confirms the negative correlation and shows the antithetic estimator.

Listing 6.7: *Antithetic variates*

```
def mc_price_antithetic(S0, K, r, sigma, T, n_paths, option="call", seed=42):
    """Use Z and -Z together; n_paths counts the total simulated paths."""
    rng = np.random.default_rng(seed)
    Z = rng.standard_normal(n_paths // 2)
    S_plus = simulate_terminal_prices(S0, r, sigma, T, None, rng, Z=Z)
    S_minus = simulate_terminal_prices(S0, r, sigma, T, None, rng, Z=-Z)
    pair_avg = 0.5 * (payoffs(S_plus, K, option) + payoffs(S_minus, K, option))
    return summarise(math.exp(-r * T) * pair_avg)


S0, K, r, sigma, T = 100.0, 100.0, 0.05, 0.20, 1.0
Z = np.random.default_rng(1).standard_normal(200_000)
Y_plus  = np.maximum(S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)  - K,
0.0)
Y_minus = np.maximum(S0 * np.exp((r - 0.5 * sigma**2) * T - sigma * math.sqrt(T) * Z) - K,
0.0)
print("correlation of payoffs from Z and -Z:", round(np.corrcoef(Y_plus, Y_minus)[0, 1], 4))
```

```
Output:
correlation of payoffs from Z and -Z: -0.4992
```

In our implementation `mc_price_antithetic`, the estimator averages $N/2$ independent pairs $(Y_i + Y_i')/2$. Because each pair is independent of every other pair, the standard error is correctly calculated as the sample standard deviation of these $N/2$ pair averages divided by $\sqrt{N/2}$:

$$\mathrm{SE}_{\mathrm{anti}} = \frac{s_{\mathrm{pair}}}{sqrtN/2}, \qquad s^2_{\mathrm{pair}} = \frac{1}{(N/2)-1}\sum_{i=1}^{N/2}\left(\frac{Y_i + Y_i'}{2} - \hat{C}_{\mathrm{anti}}\right)^2 \qquad (6.9)$$

Dividing by $\sqrt{N/2}$ on the pair standard deviation properly accounts for the internal negative covariance within each pair. In contrast, dividing the raw sample standard deviation of all N draws by $\sqrt{N}$ would incorrectly treat dependent antithetic draws as independent. Note also that the approximately $2i$ mes variance reduction observed in our experiments is specific to

this at-the-money European call setting; the exact efficiency gain varies with the nonlinearity of the payoff and option moneyness.

CHAPTER 7: IMPLEMENTATION IN PYTHON

This chapter explains how the pieces from the previous chapters are written in Python and NumPy and how they combine into the complete program. The whole code is organised as a set of small functions, one for each idea of the theory, so that the final pricer is only a few lines long. The code is available in the project folder (`mc_option_pricing.py` and `run_experiments.py`).

7.1 Environment Setup

The project is developed in Visual Studio Code with Python 3. A virtual environment keeps the dependencies of the project separate from the rest of the system. The commands below are run in the PowerShell terminal of VS Code.

Listing 7.1: *Setting up and running the project from PowerShell*

```
python -m venv .venv                  # create a virtual environment
.\.venv\Scripts\Activate.ps1         # activate it in PowerShell
pip install numpy matplotlib          # install the required libraries
python mc_option_pricing.py           # run the pricer
python run_experiments.py             # run all experiments and create the figures
```

7.2 Structure of the Code: from Concepts to Functions

Table 7.1 shows how each concept of the theory is mapped to one function in `mc_option_pricing.py`. Combining them in the order of the table gives the complete Monte Carlo pricer.

Table 7.1: Mapping between theoretical concepts and code fragments

Fragment (function)	Concept	Theory
`LCG`	Pseudo-random uniform numbers	Section 6.3
`box_muller`	Uniform → standard normal	Section 6.4
`simulate_terminal_prices`	Exact GBM solution, one step	Sections 4.5, 5.3
`simulate_gbm_paths`	Euler / log discretisation of GBM	Section 6.5
`payoffs`	Call and put payoff functions	Section 5.2
`summarise`	Mean, standard error, 95% CI	Section 6.6
`mc_price`	Discounting and complete estimator	Sections 5.3, 6.1
`black_scholes`	Analytical benchmark	Section 5.4
`mc_price_antithetic`	Variance reduction	Section 6.7

7.3 NumPy Vectorisation

A naive implementation simulates one path at a time inside a Python `for` loop. Python loops are slow because every iteration is interpreted. NumPy instead applies an operation to a whole

array at once, in compiled C code. With **vectorisation**, 100,000 or even 1,000,000 paths are simulated simultaneously. The two fragments below compute the same quantity; Table 7.2 compares their run time.

Listing 7.2: *Loop version versus vectorised NumPy version*

```
# (a) loop version: one path per iteration
total = 0.0
for _ in range(N):
    z = rng.standard_normal()
    S_T = S0 * math.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * z)
    total += max(S_T - K, 0.0)
price = math.exp(-r * T) * total / N

# (b) vectorised version: all paths at once
Z = rng.standard_normal(N)                                    # N random numbers in one call
S_T = S0 * np.exp((r - 0.5 * sigma**2) * T + sigma * math.sqrt(T) * Z)   # N prices at once
price = math.exp(-r * T) * np.maximum(S_T - K, 0.0).mean()    # N payoffs, averaged
```

Table 7.2: Run-time comparison (measured on the development computer)

Version	Time for 200,000 paths (s)	Relative speed
Python `for` loop	0.1272	1×
NumPy vectorised	0.0052	24× faster

Note: Execution times are the median of 5 `timeit` runs for 200,000 paths on Windows 11 (AMD64) (AMD64 Family 25 Model 68 Stepping 1, AuthenticAMD), Python 3.14.5, NumPy 2.5.4.

7.4 Path Generator Code

The path generator `simulate_gbm_paths` (shown in Chapter 6) translates the discrete GBM formula into array operations. `rng.standard_normal((n_paths, n_steps))` creates a matrix whose entry (i, j) is the shock Z of path i in step j. Then `np.cumprod(growth, axis=1)` multiplies the growth factors along each row, which is exactly the recursion $S_{j+1} = S_j \times (\text{growth factor})$, and the result is scaled by S_0. The first column holds S_0.

7.5 Payoff and Discounting Code

The terminal prices are generated by `simulate_terminal_prices` and the payoffs by `payoffs` (both shown earlier). The function `mc_price` combines them with discounting and the statistics of Section 6.6:

Listing 7.3: *Terminal prices and the complete Monte Carlo pricer*

```
def simulate_terminal_prices(S0, r, sigma, T, n_paths, rng, Z=None):
    """S_T = S0 * exp((r - sigma^2/2) T + sigma sqrt(T) Z), Z ~ N(0,1)."""
    if Z is None:
        Z = rng.standard_normal(n_paths)
    drift = (r - 0.5 * sigma ** 2) * T
    diffusion = sigma * math.sqrt(T) * Z
    return S0 * np.exp(drift + diffusion)



def mc_price(S0, K, r, sigma, T, n_paths, option="call", seed=42):
    """Plain Monte Carlo price of a European option."""
    rng = np.random.default_rng(seed)
    S_T = simulate_terminal_prices(S0, r, sigma, T, n_paths, rng)
    discounted = math.exp(-r * T) * payoffs(S_T, K, option)
    return summarise(discounted)
```

7.6 Putting the Fragments Together

With all fragments in place, pricing a European call and put takes only a few lines. The program prints the Monte Carlo price, its standard error, the 95% confidence interval and the Black–Scholes value.

Listing 7.4: *Main driver: price a call and a put*

```
from mc_option_pricing import mc_price, black_scholes


S0, K, r, sigma, T = 100.0, 100.0, 0.05, 0.20, 1.0
for option in ("call", "put"):
    price, se, (lo, hi) = mc_price(S0, K, r, sigma, T, n_paths=100_000, option=option)
    exact = black_scholes(S0, K, r, sigma, T, option)
    print(f"{option:>4}: MC = {price:.4f}  SE = {se:.4f}  95% CI = ({lo:.4f}, {hi:.4f})
Black-Scholes = {exact:.4f}")
```

```
Output:
call: MC = 10.4205  SE = 0.0468  95% CI = (10.3289, 10.5122)  Black-Scholes = 10.4506
 put: MC = 5.6122  SE = 0.0275  95% CI = (5.5583, 5.6661)  Black-Scholes = 5.5735
```

7.6.1 A pricer built completely from scratch

To show that no library "magic" is needed, the next fragment replaces NumPy's random generator by our own LCG and Box–Muller transform. The chain is: LCG → uniform numbers → Box–Muller → standard normals → GBM → payoff → average.

Listing 7.5: *Complete pipeline using only our own LCG and Box–Muller transform*

```
from mc_option_pricing import LCG, box_muller, simulate_terminal_prices, payoffs, summarise

gen = LCG(seed=2026)
u = gen.uniform(200_000)                          # 1. uniform numbers from our LCG
z, _ = box_muller(u[:100_000], u[100_000:])       # 2. normals from Box-Muller (100,000 of them)
ST = simulate_terminal_prices(100.0, 0.05, 0.20, 1.0, None, None, Z=z)   # 3. GBM terminal
prices
disc = math.exp(-0.05) * payoffs(ST, 100.0, "call")                    # 4. discounted payoffs
price, se, ci = summarise(disc)                                          # 5. estimate and
error
print(f"from-scratch call price = {price:.4f}  (SE {se:.4f}),  Black-Scholes = 10.4506")
```

```
Output:
from-scratch call price = 10.4093  (SE 0.0463),  Black-Scholes = 10.4506
```

7.7 Data Visualisation with Matplotlib

Visualisation helps to verify that the simulation behaves as theoretical principles predict. First, the simulated GBM trajectories are plotted across the annual horizon; Figure 7.1 visualises twenty independent sample paths.

Listing 7.6: *Plotting simulated stock-price paths*

```
import matplotlib.pyplot as plt

paths = simulate_gbm_paths(S0, r, sigma, T, 252, 20, rng, "log")
t = np.linspace(0, T, 253)
plt.plot(t, paths.T, lw=0.8)                 # one line per simulated path
plt.axhline(K, color="k", ls="--")           # strike price
plt.xlabel("Time (years)"); plt.ylabel("Stock price"); plt.show()
```

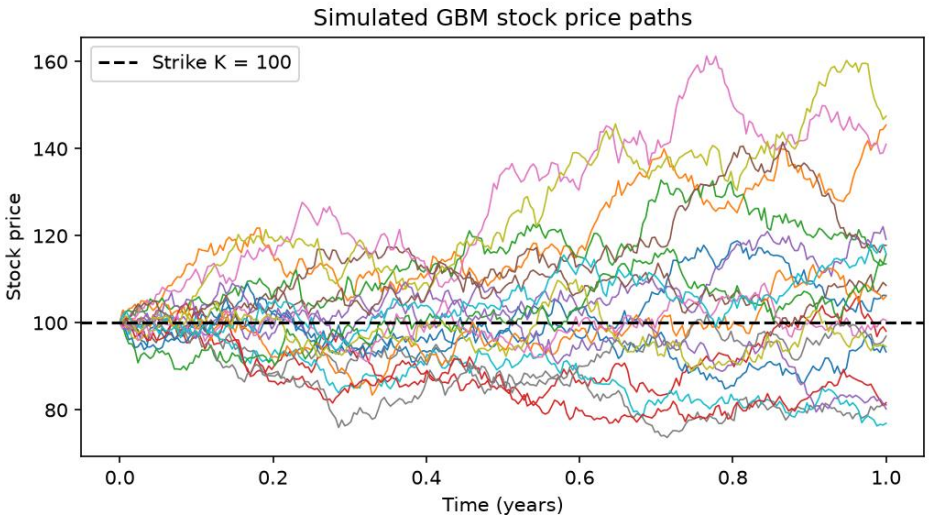


Figure 7.1: Twenty simulated GBM stock-price paths (S0 = 100, r = 5%, σ = 20%, T = 1 year).

Next, the distributions of the terminal stock price and the option payoffs are visualised to observe lognormality and the hockey-stick truncation at the strike price K.

Listing 7.7: *Plotting terminal price and payoff distributions*

```
S_T = simulate_terminal_prices(S0, r, sigma, T, 100_000, rng)
plt.hist(S_T, bins=80, density=True)          # lognormal histogram of S_T
plt.hist(payoffs(S_T, K, "call"), bins=80)    # call payoff distribution
```

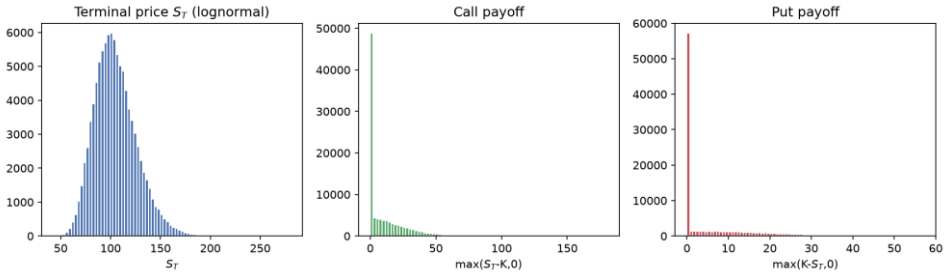


Figure 7.2: Histograms of the simulated terminal price (lognormal) and of the call and put payoffs (N = 100,000).

The terminal price is right-skewed, as expected for a lognormal variable. Both payoff histograms have a large spike at zero (a call pays nothing whenever $S_T \leq K$, which happens in about 44% of the paths) and a long tail of positive values. The price of the option is the discounted average of this distribution.

CHAPTER 8: RESULTS AND DISCUSSION

This chapter reports the numerical results obtained by running `run_experiments.py`. Unless stated otherwise the parameters are $S_0 = 100$, $K = 100$, $r = 5$, $\sigma = 20$ and $T = 1$ year, with a fixed random seed so that every number can be reproduced.

8.1 Convergence to the Black–Scholes Price

Figure 8.1 (left) shows the Monte Carlo estimate of the call price as the number of paths grows from 100 to one million. The estimate fluctuates strongly for small N and settles on the Black–Scholes value for large N, which illustrates the Law of Large Numbers. The right-hand plot shows that the standard error falls parallel to the dashed $1/\sqrt{N}$ line.

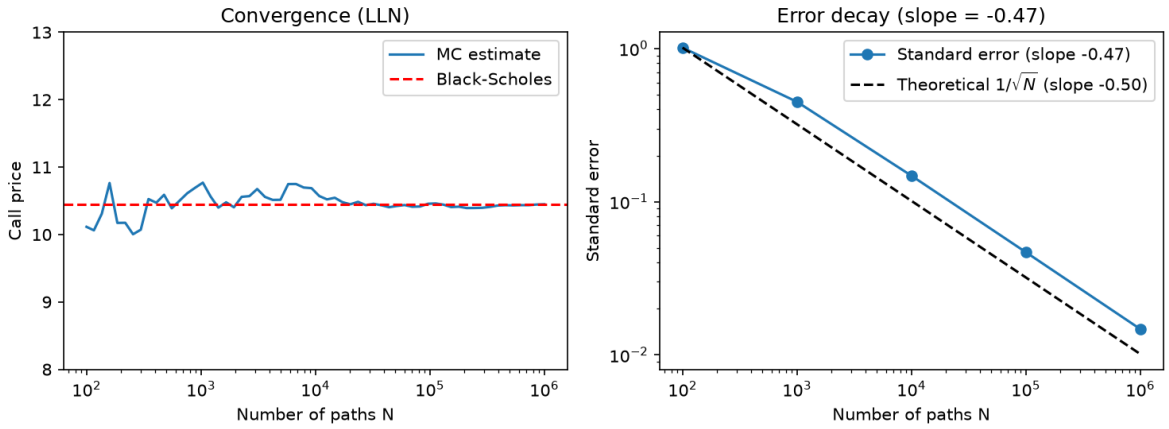


Figure 8.1: Convergence of the call price estimate (left) and decay of the standard error (right).

Table 8.1: Call price versus number of paths (Black–Scholes value = 10.4506)

N (paths)	MC call price	Standard error	95% confidence interval	|MC − BS|
100	7.8258	1.0120	(5.842, 9.809)	2.6248
1,000	9.9120	0.4483	(9.033, 10.791)	0.5386
10,000	10.3452	0.1477	(10.056, 10.635)	0.1054
100,000	10.4205	0.0468	(10.329, 10.512)	0.0300
1,000,000	10.4532	0.0147	(10.424, 10.482)	0.0026

For $N = 100$, the 95% confidence interval is $(5.842, 9.809)$, which does not contain the true Black–Scholes price of 10.4506. At such a small sample size, the payoff distribution is severely right-skewed with a large point mass at zero (over 44% of calls expire worthless), rendering the asymptotic normality assumption of the Central Limit Theorem unreliable. Furthermore, a 95% confidence interval has an expected non-coverage rate of 5%, so an occasional miss is statistically expected by chance. Section 8.3 formally evaluates confidence interval coverage across 1,000 independent replications.

Increasing N by a factor of 100 (from 1,000 to 100,000 paths) reduced the standard error from 0.4483 to 0.0468, i.e. by a factor of 9.6, close to the theoretical factor of $\sqrt{100} = 10$. With one million paths the estimate 10.4532 differs from the exact price by only 0.0026.

A linear regression of $\log_{10}(\text{SE})$ against $\log_{10}(N)$ across the five data points yields a fitted slope of **-0.466** ($R^2 = 0.996$), in close agreement with the theoretical slope of -0.50 corresponding to the $1/\sqrt{N}$ convergence rate.

8.2 Main Result: European Call and Put

Table 8.2: Monte Carlo prices with N = 100,000 paths

Option	MC price	Std. error	95% CI	Black–Scholes	BS inside CI?
Call	10.4205	0.0468	(10.3289, 10.5122)	10.4506	Yes
Put	5.6122	0.0275	(5.5583, 5.6661)	5.5735	Yes

For the call, the Monte Carlo estimate is 10.4205 against an exact price of 10.4506; for the put, 5.6122 against 5.5735. In both cases the Black–Scholes price lies inside the 95% confidence interval, so the simulation agrees with the analytical benchmark within its statistical error. The put is cheaper than the call because the interest rate is positive: the forward price of the stock is above the strike.

8.3 Validity of the Confidence Interval

A 95% confidence interval should contain the true value in about 95% of repeated experiments. To test this, the call was priced 1000 times with 1000 paths each and different random seeds, and we counted how often the Black–Scholes price was inside the interval. The observed coverage was **95.3%**. The standard error of this empirical proportion over $M = 1{,}000$ trials is $\sqrt{p(1-p)/M} \approx 0.67$, giving an empirical 95% confidence interval for the coverage itself of [94.0\%, 96.6\%]. The theoretical 95% benchmark falls well inside this interval, confirming that the CLT-based interval is statistically calibrated.

8.4 Different Strike Prices

Table 8.3: Monte Carlo (N = 100,000) versus Black–Scholes across various strikes

Strike K	Call MC	Call SE	Call BS	Call in CI?	Put MC	Put SE	Put BS	Put in CI?
80	24.5348	0.0608	24.5888	Yes	0.7019	0.0086	0.6872	Yes
90	16.6562	0.0552	16.6994	Yes	2.3356	0.0171	2.3101	Yes
100	10.4205	0.0468	10.4506	Yes	5.6122	0.0275	5.5735	Yes
110	6.0397	0.0370	6.0401	Yes	10.7437	0.0380	10.6753	Yes

Strike K	Call MC	Call SE	Call BS	Call in CI?	Put MC	Put SE	Put BS	Put in CI?
120	3.2669	0.0277	3.2475	Yes	17.4831	0.0470	17.3950	Yes

Note: All five strike prices are priced using the same simulated terminal stock prices (common random numbers). Consequently, the simulation errors across strikes are strongly correlated rather than independent replications. For strikes $K \leq 100$, all simulated call prices fall slightly below their Black–Scholes values, reflecting the identical small downward sampling deviation in the simulated mean terminal stock price $\overline{S}_T$.

For every strike, the analytical Black–Scholes price falls within the 95% confidence interval of the Monte Carlo estimate, confirming statistical consistency across the full range of moneyness. Calls become cheaper and puts more expensive as the strike increases, exactly as the payoff functions predict. The relative error is largest for deep out-of-the-money options (for example the put with $K = 80$), where only a small fraction of the paths contribute a non-zero payoff.

8.5 Effect of the Time Step (Euler versus Log Scheme)

Table 8.4: Call price for different numbers of time steps (200,000 paths; exact value 10.4506)

Time steps M	Euler–Maruyama	Log (exact) scheme
1	10.2367	10.4897
5	10.4575	10.4861
20	10.4121	10.4166
100	10.4378	10.4392
252	10.4774	10.4776

The log scheme is insensitive to the number of steps, as it reproduces the exact distribution at every step; the remaining differences between rows are sampling noise. The Euler scheme with a single step is visibly biased (10.237 compared with 10.451), and the bias becomes small as the number of steps increases. For a plain European option the one-step exact formula is therefore preferable, and the multi-step schemes are needed only for path-dependent payoffs.

8.6 Variance Reduction by Antithetic Variates

Table 8.5: Plain versus antithetic Monte Carlo for the call option

N (payoffs)	Plain: price	Plain: SE	Antithetic: price	Antithetic: SE	Variance ratio
1,000	9.9120	0.4483	10.0004	0.3136	2.04
10,000	10.3452	0.1477	10.4101	0.1049	1.98
100,000	10.4205	0.0468	10.4673	0.0331	2.00

For the same total number of payoffs, antithetic sampling reduces the standard error by about 29% (from 0.0468 to 0.0331 at $N = 100{,}000$). The variance ratio is close to 2 in every row, demonstrating that antithetic sampling effectively doubles computational efficiency. As established in Section 6.7, the standard error is evaluated on the $N/2$ independent pair averages, and this $\approx 2\times$ efficiency gain is specific to the at-the-money payoff structure.

8.7 Multi-Seed Assessment

To confirm that the single-seed results in Table 8.1 and Table 8.2 are typical and not an artifact of a fortunate seed, the main simulation ($N = 100{,}000$) was repeated over 200 distinct random seeds. For the call option (Black–Scholes price 10.4506), the mean estimate across seeds is **10.4509** with an empirical standard deviation across seeds of **0.0473**, closely matching the analytical standard error of 0.0468. Exactly 190 out of 200 confidence intervals (**95.0%**) covered the true Black–Scholes price.

For the put option (Black–Scholes price 5.5735), the mean estimate across seeds is **5.5746** with an empirical standard deviation of **0.0298** (analytical standard error 0.0275) and empirical coverage of **95.0%**. These multi-seed results verify that the single-seed estimate is representative and confirm that the analytical standard error formula accurately quantifies sampling variability.

8.8 Parameter Sensitivity: Volatility and Maturity

To examine how market parameters influence pricing accuracy, the Monte Carlo call pricer was evaluated across a 3×3 grid of volatility ($\sigma \in \{10\ ,20\ ,40\ \}$) and maturity ($T \in \{0.25,1.0,2.0\}$ years) at $S_0 = K = 100$ and $r = 5\ $, with $N = 100{,}000$ paths.

Table 8.6: Sensitivity of Monte Carlo call price and standard error to volatility and maturity (N = 100,000)

Volatility σ	Maturity T (yr)	MC Call	Std. Error	95% CI	Black–Scholes	BS in CI?
10%	0.25	2.6556	0.0108	(2.634, 2.677)	2.6648	Yes
10%	1.00	6.7844	0.0245	(6.736, 6.833)	6.8050	Yes
10%	2.00	11.3811	0.0376	(11.307, 11.455)	11.4126	Yes
20%	0.25	4.6000	0.0210	(4.559, 4.641)	4.6150	Yes
20%	1.00	10.4205	0.0468	(10.329, 10.512)	10.4506	Yes
20%	2.00	16.0858	0.0716	(15.945, 16.226)	16.1268	Yes
40%	0.25	8.5330	0.0431	(8.449, 8.618)	8.5526	Yes
40%	1.00	18.0072	0.0998	(17.812, 18.203)	18.0230	Yes
40%	2.00	26.3057	0.1592	(25.994, 26.618)	26.2902	Yes

Across all nine parameter combinations, the analytical Black–Scholes price lies strictly within the 95% confidence interval. Crucially, the table demonstrates how the standard error scales with volatility: as σ doubles from 10% to 20% to 40% at $T = 1.0$ year, the standard error increases from 0.0245 to 0.0468 to 0.0998 — an almost exact doubling at each step. This occurs because higher volatility disperses the terminal stock price distribution S_T, expanding the spread of the payoff $\max(S_T - K, 0)$. Consequently, contracts written on highly volatile assets require substantially larger sample sizes N to achieve the same absolute dollar precision.

8.9 Sanity Checks

- **Terminal price mean and put–call parity:** The simulated mean terminal price is $\overline{S}_T = 105.055$, compared with the theoretical expectation $\mathrm{E}[S_T] = S_0 e^{rT} = 105.127$. Because calls and puts were priced with common random numbers, the put–call parity gap $(C - P) - (S_0 - Ke^{-rT}) = 4.8083 - 4.8771 = -0.0687$ identically equals the discounted deviation of the simulated stock price, $e^{-rT}(\overline{S}_T - S_0 e^{rT}) = e^{-0.05}(105.055 - 105.127) = -0.0687$. Both checks therefore reflect the same sampling deviation of $\overline{S}_T$, whose discounted standard error is $\mathrm{sd}(S_T)e^{-rT}/\sqrt{N} \approx 0.064$, where $\mathrm{sd}(S_T) = S_0 e^{rT}\sqrt{e^{\sigma^2 T} - 1} \approx 21.2$. The gap is approximately 1.1 standard errors, consistent with sampling noise.
- **Probability of finishing in the money:** the simulation gives 0.5575; the risk-neutral value $\Phi(d_2)$ is 0.5596.
- **Random numbers:** the LCG output has mean 0.4987 and variance 0.0835 (ideal values 0.5 and 0.0833), and the Box–Muller normals have mean 0.0144 and standard deviation 1.0006.
- **Speed:** the vectorised NumPy implementation is about 24 times faster than the loop version.

8.10 Discussion and Limitations

The experiments show that Monte Carlo simulation is an accurate and easy-to-implement pricing method: its estimates agree with the analytical solution and its error can be measured and controlled. Its main drawback is slow convergence: to gain one additional decimal digit, 100 times more paths are required. For a plain European option, the Black–Scholes formula is obviously faster and exact, so Monte Carlo has no practical advantage here; its value in this project is that the same program extends with almost no change to options for which no formula exists. Other limitations are the model assumptions (constant volatility and interest rate, no

dividends, no jumps), which real markets do not satisfy, and the fact that the results are random and depend on the seed.

CHAPTER 9: CONCLUSION

In this project we studied and implemented the Monte Carlo pricing of European call and put options. The report first reviewed the necessary mathematical background — probability and statistics, the central limit theorem, random walks, Brownian motion, the lognormal distribution, Geometric Brownian Motion and an intuitive form of Itô's lemma — and the financial mathematics of continuous compounding, option payoffs, risk-neutral valuation and the Black–Scholes formula. It then connected each concept to a Python fragment and combined the fragments into a complete, vectorised NumPy pricer.

The main findings are:

- The Monte Carlo prices with $N = 100{,}000$ (10.4205 for the call and 5.6122 for the put) agree with the Black–Scholes prices (10.4506 and 5.5735) within their 95% confidence intervals.
- The standard error decreases proportionally to $1/\sqrt{N}$, as predicted by the Central Limit Theorem, and the confidence intervals have the correct coverage (95.3% observed for a nominal 95%).
- A pricer built entirely from our own LCG and Box–Muller transform reproduces the benchmark price, showing that the full chain from random numbers to option price is understood.
- Vectorisation with NumPy makes the simulation about 24 times faster than a Python loop, and antithetic variates halve the variance at no extra cost.

The project therefore achieved its objectives: it provides a clear, validated implementation together with the conceptual understanding behind every step.

CHAPTER 10: FUTURE SCOPE

The present work is limited to plain European options under constant volatility. The following extensions are natural continuations once the corresponding theory has been studied:

- **Path-dependent options:** Asian options (average price), barrier options and lookback options, for which no simple formula exists and the multi-step path generator of Chapter 7 is essential.
- **More realistic models:** stochastic volatility (Heston model) and jump-diffusion (Merton model, built on the Poisson process), and time-dependent volatility and interest rates.
- **Further variance reduction:** control variates (using the known mean of the stock price or the Black–Scholes price of a related option), importance sampling and stratified sampling.
- **Quasi-Monte Carlo:** low-discrepancy sequences (Sobol, Halton), which can reach an error close to $1/N$.
- **Sensitivities (the Greeks):** estimating delta, gamma and vega by finite differences, pathwise derivatives or the likelihood-ratio method.
- **American options:** the Longstaff–Schwartz least-squares Monte Carlo algorithm.
- **Multi-asset options and dividends:** basket options using correlated Brownian motions, and stocks paying dividends.
- **Performance and tools:** parallel execution, GPU acceleration, and a graphical interface for entering parameters and viewing results.

REFERENCES

1. Black, F., & Scholes, M. (1973), "The pricing of options and corporate liabilities," *Journal of Political Economy*, 81(3), 637–654.
2. Merton, R. C. (1973), "Theory of rational option pricing," *Bell Journal of Economics and Management Science*, 4(1), 141–183.
3. Boyle, P. P. (1977), "Options: A Monte Carlo approach," *Journal of Financial Economics*, 4(3), 323–338.
4. Hull, J. C. (2018), *Options, Futures, and Other Derivatives* (10th ed.), Pearson.
5. Glasserman, P. (2004), *Monte Carlo Methods in Financial Engineering*, Springer.
6. Ross, S. M. (2014), *Introduction to Probability Models* (11th ed.), Academic Press.
7. Shreve, S. E. (2004), *Stochastic Calculus for Finance II: Continuous-Time Models*, Springer.
8. Box, G. E. P., & Muller, M. E. (1958), "A note on the generation of random normal deviates," *The Annals of Mathematical Statistics*, 29(2), 610–611.
9. Lehmer, D. H. (1951), "Mathematical methods in large-scale computing units," *Proceedings of the 2nd Symposium on Large-Scale Digital Calculating Machinery*, Harvard University Press, 141–146.
10. Metropolis, N., & Ulam, S. (1949), "The Monte Carlo method," *Journal of the American Statistical Association*, 44(247), 335–341.
11. Kloeden, P. E., & Platen, E. (1992), *Numerical Solution of Stochastic Differential Equations*, Springer.
12. Harris, C. R., et al. (2020), "Array programming with NumPy," *Nature*, 585, 357–362.
13. Hunter, J. D. (2007), "Matplotlib: A 2D graphics environment," *Computing in Science & Engineering*, 9(3), 90–95.
14. Hammersley, J. M., & Morton, K. W. (1956), "A new Monte Carlo technique: antithetic variates," *Mathematical Proceedings of the Cambridge Philosophical Society*, 52(3), 449–475.
15. Wilmott, P., Howison, S., & Dewynne, J. (1995), *The Mathematics of Financial Derivatives: A Student Introduction*, Cambridge University Press.
16. Feller, W. (1968), *An Introduction to Probability Theory and Its Applications, Volume 1* (3rd ed.), Wiley.
17. Knuth, D. E. (1997), *The Art of Computer Programming, Volume 2: Seminumerical Algorithms* (3rd ed.), Addison-Wesley.
18. O'Neill, M. E. (2014), "PCG: A family of simple fast space-efficient statistically good algorithms for random number generation," *Technical Report HMC-CS-2014-0905*, Harvey Mudd College.
19. Boyle, P., Broadie, M., & Glasserman, P. (1997), "Monte Carlo methods for security pricing," *Journal of Economic Dynamics and Control*, 21(8–9), 1267–1321.